Chapter 5 – Polymorphism

(*Object-Oriented Programming*)

Outline

5.1 Type Casting

5.2 Early Binding and Late Binding

5.3 Introduction to Polymorphism

5.4 Virtual Functions and Pure Virtual Functions

5.5 Abstract Class

5.1 Type Casting

- Type casting: conversion of a value from one type to another.
- Two main categories:
  - Implicit casting
  - Explicit casting
- Also important in OOP:
  - Casting between base and derived classes

Implicit Type Casting

- Performed automatically by the compiler
- Occurs when assigning between compatible types
- Safe conversion
  - No data loss
  - Value is preserved
  - Also called automatic conversion
- Example – Safe conversion

```
int a = 10;
double b = a;
```

Implicit Type Casting

- Potential Data Loss: occurs when converting
  - from larger → smaller type
  - more precise → less precise type
- Example – Potential Data Loss

```
double x = 9.8;
int y = x;  // double → int, y = 9
```

- In this case, explicit casting should be used to make the program clearer, as it indicates that the conversion is intentional.

Explicit Type Casting

- Performed manually by the programmer
- Required when conversion is not safe or not automatic
- Controlled and intentional type conversion
- Why use explicit casting?
  - Makes the programmer’s intent clear and visible
  - Helps avoid unintended implicit conversions
  - Improves code readability and maintainability

Explicit Type Casting

- C++ functional-style casting

```
int a = 5, b = 2;
double result = a / b;  // result = ?
```

→ use explicit casting

```
double result = double(a) / b; // result = 2.5
```

- C++ provides an alternative casting style that is more informative:

1. static_cast
   - Used for standard type conversions
   - Checked at compile time

```
int a = 10;
double b = static_cast<double>(a);
```

Explicit Type Casting

2. dynamic_cast
   - Used for casting within inheritance hierarchies
   - Checked at runtime

```
Base* b = new Derived();
Derived* d = dynamic_cast<Derived*>(b);
```

3. const_cast
   - Used to add/remove const qualifier

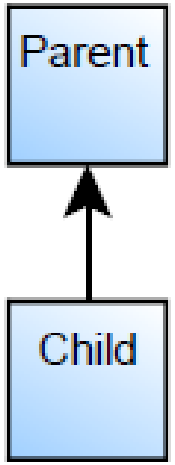
```
const int x = 10;
int* p = const_cast<int*>(&x);
```

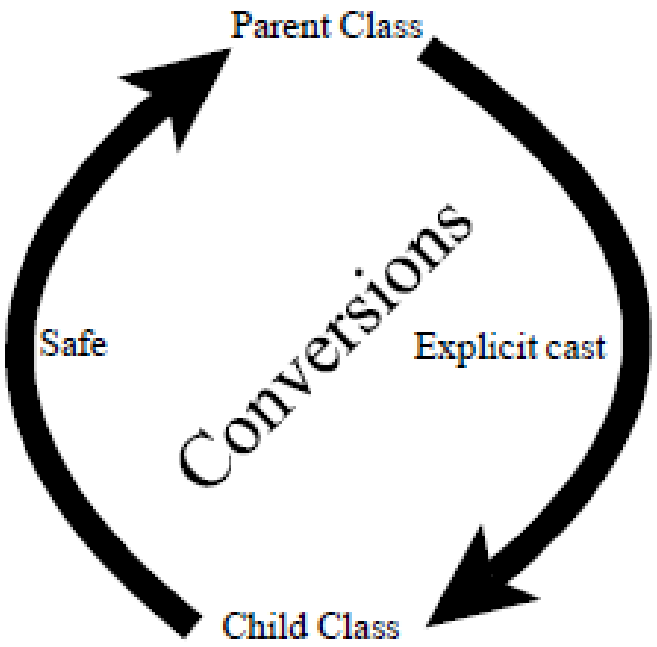
4. reinterpret_cast
   - Used for low-level, unsafe conversions

```
int x = 65;
char* c = reinterpret_cast<char*>(&x);
```

Casting in Inheritance

- Occurs between base class and derived class
- Upcasting (Derived → Base)
  - Implicit, safe
  - Supports polymorphism
- Downcasting (Base → Derived)
  - Requires explicit cast
  - Use dynamic_cast for safety





Example 5.1

```cpp
//Shape.h
#pragma once
#include <iostream>
#include <string>
using namespace std;

class Shape
{
protected:
    string color;
public:
    Shape(string c = "White") : color(c) {}

    string getColor() const { return color; }
};
```

```cpp
//Circle.cpp
#pragma once
#include "Shape.h"
class Circle : public Shape
{
private:
    float radius;
public:
    Circle(string c = "White", float r = 1)
        : Shape(c), radius(r > 0 ? r : 1)
    {}

    float getRadius() const { return radius; }
};
```

Example 5.1

```
//main.cpp
#include "Shape.h"
#include "Circle.h"

int main()
{
    Shape s1("Blue"), s2;
    Circle c1("Red", 2);

    s2 = s1; // assign two objects of the same type
    cout << s2.getColor() << endl;

    s1 = c1; // assign a derived class object to a base class object
    cout << s1.getColor() << endl;
    //cout << s1.getRadius() << endl;

    return 0;
}
```

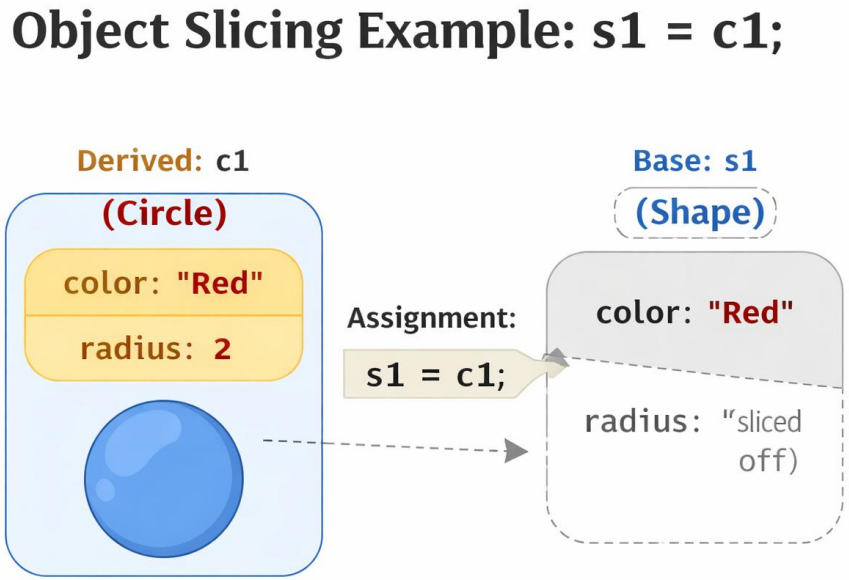
Error: The Shape class does not have the method getRadius()

Casting in Inheritance

- An assignment between objects of different types in an inheritance relationship.

```
s1 = c1;
```

- An implicit upcasting from Circle to Shape.
- It is handled automatically by the compiler.
- However, it also causes object slicing.
  - Only the Shape part of c1 is copied into s1
  - The Circle-specific data (e.g., radius) is lost



Example 5.2

```
//main.cpp
#include "Shape.h"
#include "Circle.h"

int main()
{
    Shape s("Blue");
    Circle c("Red", 2);

    // c = s; //assign a base class object to a derived class object ->
    Invalid

    return 0;
}
```

Error: binary '=': no operator found which takes a right-hand operand of type 'Shape' (or there is no acceptable conversion)

Casting in Inheritance

- Cannot implicitly assign a base class object to a derived class object

```
c = s; //invalid
```

- Solution: using conversion constructor or assignment operator

Example 5.2

```
//Circle.cpp
#pragma once
#include "Shape.h"
class Circle : public Shape
{
private:
    float radius;
public:
    Circle(string c = "White", float r = 1): Shape(c), radius(r > 0 ? r : 1) {}

    //Option 1: conversion constructor
    Circle(const Shape& s) : Shape(s), radius(1) {}

    //Option 2: assignment operator
    Circle& operator=(const Shape& s) {
        this->color = s.getColor();
        this->radius = 1;
        return *this;
    }
    float getRadius() const { return radius; }
};
```

Example 5.2

```
//main.cpp
#include "Shape.h"
#include "Circle.h"

int main()
{
    Shape s("Blue");
    Circle c("Red", 2);

    c = s; //OK - assign a base class object to a derived class object
    cout << c.getColor() << endl;
    cout << c.getRadius() << endl;

    return 0;
}
```

Example 5.3

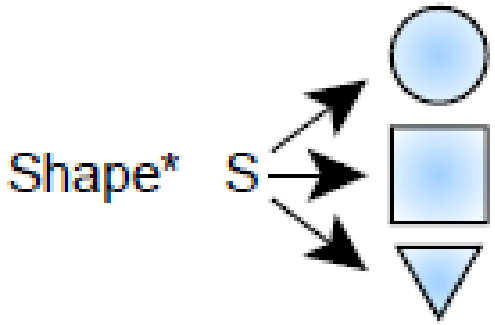
```
//main.cpp
#include "Shape.h"
#include "Circle.h"

int main()
{
    Shape s("Blue");
    Circle c("Red", 2);

    // Upcasting: base class pointer points to a derived class object
    Shape* ps = &c; // safe
    cout << ps->getColor() << endl;

    // Downcasting: derived class pointer points to a base class object
    Circle* pc = (Circle*)&s; // unsafe
    cout << pc->getColor() << endl;
    cout << pc->getRadius() << endl; // radius = ???

    return 0;
}
```

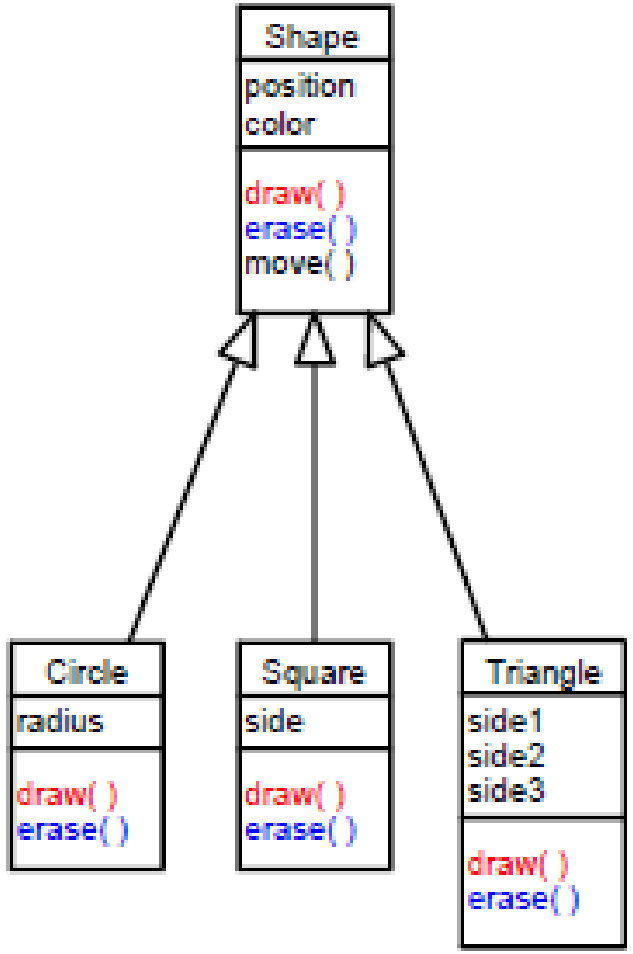
5.2 Early Binding and Late Binding

- The process of matching a specific function call to its respective function definition is known as **binding**.
- Two types:
  - Early Binding (Static Binding)
  - Late Binding (Dynamic Binding)

Early Binding

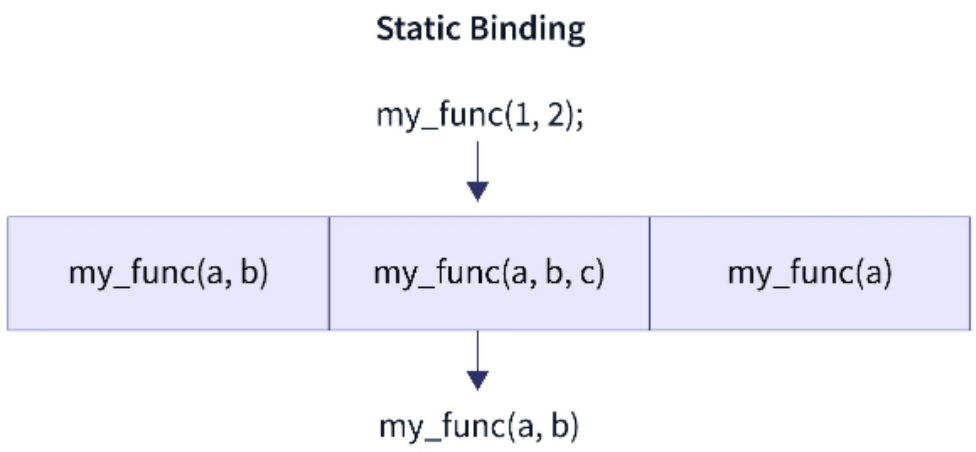
- Binding occurs at compile time
- Member function calls through an object pointer are resolved **at compile time**
- Characteristics:
  - Faster execution
  - No runtime overhead





Early Binding

- Static binding in C++ is implemented using function overloading & operator overloading.



Example 5.4

```cpp
//Shape.h
#pragma once
#include <iostream>
#include <string>
using namespace std;

class Shape
{
protected:
    string color;
public:
    Shape(string c = "White") : color(c) {}
    string getColor() const {  return color; }
    void draw() {
        cout << "This is a Shape" << endl;
    }
};
```

```cpp
//Circle.cpp
#pragma once
#include "Shape.h"
class Circle : public Shape
{
private:
    float radius;
public:
    Circle(string c = "White", float r = 1)
        : Shape(c), radius(r > 0 ? r : 1)
    {}
    float getRadius() const { return radius; }
    // override
    void draw() {
        cout << "This is a Circle" << endl;
    }
};
```

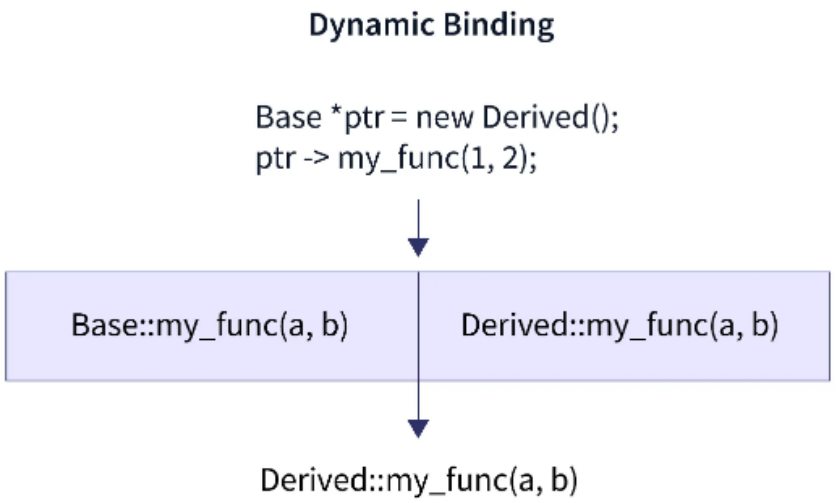
Example 5.4

```
//main.cpp
#include "Shape.h"
#include "Circle.h"

int main()
{
    Shape* s;

    s = new Shape("Blue");
    s->draw(); // Which draw() is called?

    s = new Circle("Red", 2);
    s->draw(); // Which draw() is called?

    return 0;
}
```

* **Uses early binding,** No polymorphism

Late Binding

- There are instances in our program when the compiler cannot get all the information at compile time to resolve a function call.
- Member function calls through an object pointer are resolved **at runtime**, depending on the actual object being referenced.
- It is very flexible as one function is used to handle different objects.
- Requires:
  - Inheritance
  - Virtual functions
  - Base class pointer/reference



5.3 Introduction to Polymorphism

- Polymorphism is the property of the same object to behave differently in different contexts given the same message.
- Compile time (static) polymorphism:
  - Function or operator overloading
- Run–time (dynamic) polymorphism:
  - Method Overriding and Virtual Functions
  - The exact method binding is determined at runtime based on the actual object being instantiated.
  - Polymorphism is achieved only by using pointers (or references) to the base class. It is not possible using objects.

5.4 Virtual Functions

- A virtual function is a member function declared in a base class using the virtual keyword and overridden in a derived class.
- It enables run-time polymorphism based on the actual object type, not pointer type.
- Virtual function = same interface, different behavior at runtime.
- Characteristics:
  - Enables dynamic binding.
  - Must be called via base class pointer or reference.
- Syntax:

```
virtual Return_Type functionName([Parameter_List])
```

Virtual Functions

- Mechanism:
  - A virtual table (vtable) is created additionally to store the pointers to all virtual functions available to all the objects of the class.
  - A single pointer to the virtual table is inserted in all class objects when the class contains at least one virtual function.
  - Whenever an object of such class wants to execute a virtual function, the table is referred to and the appropriate function is called.
- Once a function is declared virtual, it remains virtual all the way down the inheritance hierarchy.
- There are no virtual constructors, but destructors can be declared as virtual.

Example 5.5

```
//Shape.h
#pragma once
#include <iostream>
#include <string>
using namespace std;

class Shape
{
protected:
    string color;
public:
    Shape(string c = "White") : color(c) {}
    string getColor() const {  return color; }
    virtual void draw() {
        cout << "This is a Shape" << endl;
    }
};
```

Example 5.5

```
//main.cpp
#include "Shape.h"
#include "Circle.h"

int main()
{
    Shape* s;

    s = new Shape("Blue");
    s->draw(); // Call the draw() function of the Shape class

    s = new Circle("Red", 2);
    s->draw(); // call the draw() function of the Circle class

    return 0;
}
```

Example 5.6

```
class Parent
{
   public:
                void    funcA( );
   virtual      void    funcB( );
                void    funcC( );
};

class Child : public Parent
{
   public:
                void    funcA( );
   virtual      void    funcB( );
};
```

```
int  main( )
{
   Parent*  P1 = new  Parent;
   Parent*  P2 = new  Child;
   Child*   C = new  Child;

   P1->funcA( );    //_____________________
   P1->funcB( );    //_____________________
   P1->funcC( );    //_____________________
   P2->funcA( );    //_____________________
   P2->funcB( );    //_____________________
   P2->funcC( );    //_____________________
   C->funcA( );     //_____________________
   C->funcB( );     //_____________________
   C->funcC( );     //_____________________
```

Default Arguments to Virtual Functions

- Default arguments are bound at compile time
- Virtual functions are resolved at runtime

→ This difference can lead to **unexpected behavior.**

```
class Base {
public:
    virtual void display(int x = 10) {
        cout << "Base: " << x << endl;
    }
};
```

```
class Derived : public Base {
public:
    void display(int x = 20) override {
        cout << "Derived: " << x << endl;
    }
};
```

```
int main()
{
    Base* b = new Derived();
    b->display(); // Output ???
}
```

Why do we need a virtual destructor?

- In polymorphism, objects are often accessed via base class pointers.
- Deleting such objects requires proper cleanup of both:
  - Derived class resources
  - Base class resources

Problem without virtual destructor

```
class Shape {
public:
    ~Shape() {
        cout << "Shape destructor" << endl;
    }
};

class Circle : public Shape {
public:
    ~Circle() {
        cout << "Circle destructor" << endl;
    }
};

int main()
{
    Shape* s = new Circle();
    delete s;
}
```

Output

```
Shape destructor
```

Problem

- Circle destructor is not called.
- Derived class resources are not released → Memory leaks.
- Violates correct object destruction order.

Solution – Virtual Destructor

```
class Shape {
public:
    virtual ~Shape() {
        cout << "Shape destructor" << endl;
    }
};

class Circle : public Shape {
public:
    ~Circle() {
        cout << "Circle destructor" << endl;
    }
};

int main()
{
    Shape* s = new Circle();
    delete s;
}
```

Output

```
Circle destructor
Shape destructor
```

Pure Virtual Functions

- A pure virtual function is a virtual function with no implementation in the base class.
- Syntax:

```
virtual Return_Type functionName([Parameter_List]) = 0;
```

- Forces derived classes to implement.
- Pure virtual function = interface definition.
- Ensures all derived classes follow the same structure.
- Foundation of abstract design in OOP.

Pure Virtual Functions

```
//Shape.h
#pragma once
#include <iostream>
#include <string>
using namespace std;

class Shape
{
protected:
    string color;
public:
    Shape(string c = "White") : color(c) {}
    string getColor() const {  return color; }
    virtual void draw() {
        cout << "This is a Shape" << endl;
    }
    virtual float area() = 0;
};
```

Pure Virtual Functions

```
//Circle.cpp
#pragma once
#include "Shape.h"
class Circle : public Shape
{
private:
    float radius;
public:
    Circle(string c = "White", float r = 1)
        : Shape(c), radius(r > 0 ? r : 1)
    {}
    float getRadius() const { return radius; }
    // override
    void draw() {
        cout << "This is a Circle" << endl;
    }
    float area() override {
        return 3.14 * radius * radius;
    }
};
```

5.5 Abstract Class

- An abstract class is a class that has at least one pure virtual function.
- Derived classes must **override** all pure virtual functions, otherwise still abstract.
- It is **not possible to create objects** of an abstract class.
- **Pointers or references** of abstract class type can be declared.

```
int main()
{
    //Shape s; -> error, Shape is an abstract class

    Shape* s; //ok
    s = new Circle("Red", 2);
    cout<< s->area() <<endl;

    return 0;
}
```